

搭建系统实战指南

一、简介与环境搭建

是目前最流行的应用开发框架，它提供了一套完整的工具链来构建系统。核心概念包括：（文档加载器）、（文本切分器）、（嵌入模型）、（向量存储）和（调用链）。的价值在于将这些组件标准化，开发者可以像搭积木一样组合不同的组件，快速搭建出原型系统。

环境搭建非常简单。首先安装必要的包：

。如果使用中文嵌入模型，还需要安装。建议使用或更高版本，并创建虚拟环境来隔离依赖。另外，需要准备一个的推荐用（性价比高、中文能力强）或作为入门。

二、从零搭建流水线

搭建的第一步是加载文档。提供了丰富的文档加载器，覆盖了（）、（）、网页（）、（）等常见格式。以下载一个为例：

，然后
你的文件，
即可获得文档对象列表。每页会变成一个对象，里面包含（页面文本）和（如页码、来源文件名等）。

第二步是文档切分。使用即可满足大多数场景：

，然后。
是每块的最大字符数，是相邻块之间的重叠字符数。对于中文内容，建议设置为到，设为到。切分后的每个块依然是一个对象。

第三步是生成嵌入并存入向量库。以 `openai/text-embedding-ada-002` 的中文嵌入模型为例：

`python openai.Embedding.create(input=[text])`，然后 `embedding[0]['embedding']`。这一步是最耗时的，因为需要将每个文本块都通过嵌入模型转换成向量。对于几百个块来说，上可能需要几十秒到一两分钟；但如果块数量很大，建议使用 `openai.AsyncEmbeddingClient` 加速。

最后一步是构建问答链。创建一个提示词模板，将检索到的上下文和用户问题填入模板，再发送给 `openai.GPT3`。

请根据以下参考资料回答问题：
`context`，然后用 `openai.GPT3` 构建链，最后调用

即可获得回答。

三、进阶技巧：检索器调优

检索器是整个系统的性能瓶颈，它的质量直接决定了最终答案的上限。调优检索器需要从多个维度入手。首先是控制返回数量（参数 `k`）：`k` 太小会导致信息不足，`k` 太大则会引入噪声。推荐从 `k=5` 开始尝试，根据实际效果调整。其次是相似度阈值过滤：设置一个最小相似度分数，低于该分数的结果直接丢弃，避免被不相关的内容干扰。

元数据过滤（`metadata_filtering`）是另一个重要工具。在存入向量库时，为每个文档块附加元数据信息（如章节标题、文档来源、创建日期等）。查询时可以根据元数据进行预过滤，比如只搜索某一章节的内容，或者只检索特定时间范围内的文档。

`OpenSearch` 的支持在 `OpenSearch` 时传入 `filter` 参数来实现这个功能。

查询改写（`query rewriting`）是提升检索质量的重要技巧。用户的原始问题往往不够精确，直接拿去检索效果不佳。可以在检索之前，让 `openai.GPT3` 将原始问题改写为更具体、更适合检索的形式。例如，用户问「它怎么工作的？」，结合对话历史改写为「`context` 中的自注意力机制是如何工作的？」。此外，还可以将一个复杂问题拆解为多个子问题，分别检索后再汇总答案。

四、多轮对话与记忆管理

基础 `openai.GPT3` 只能处理单轮问答，但在实际应用中，用户往往会进行追问。要实现多轮对话，需要引入记忆组件。`openai.memory` 提供了

`Memory`，它会保存完整的对话历史。在每轮问答时，将历史对话也作为上下文的一部分，这样模型就能理解用户追问的内容。但需要注意的是，不能直接把整个对话历史拼到向量查询里，那会把闲聊内容也带进去，干扰检索结果。

更好的做法是分离查询和上下文。每次对话时，先用 `llm` 结合历史对话将用户当前的追问改写为一个独立的、自包含的查询语句，然后用这个改写后的查询去做向量检索。拿到检索结果后，再把历史对话、检索结果和当前问题一起发给 `llm` 生成回答。这样检索过程不会受到历史对话噪音的影响，而生成过程仍然可以利用历史上下文。

五、常见问题与优化思路

第一个常见问题是检索结果与问题不相关。可能的原因包括：文档切分太大或太小、嵌入模型不适合当前领域、向量检索阈值设得太低。排查方法是逐个检查检索返回的文档块内容，判断是切分出了问题还是嵌入模型不够好。如果发现是嵌入模型的问题，可以考虑换用更大规模的模型或针对你的领域进行微调。

第二个常见问题是 `llm` 忽略了检索到的上下文，凭空编造内容（幻觉）。这通常是因为提示词模板写得不够严格。需要明确告诉模型「仅根据参考资料回答，如果资料中没有找到相关信息，请直接说明」，并在模板中把检索到的上下文放在显眼位置。适当降低 `llm` 的温度参数（`temperature` 到 `0.1`）也能让模型更忠实于给定的上下文。

第三个常见问题是响应速度太慢。`llm` 系统的延迟主要来自三个环节：嵌入查询（约 `10ms` 到 `100ms`）、向量检索（约 `10ms` 到 `100ms`）和 `llm` 生成（约 `10ms` 到 `100ms`）。可以通过使用更小的嵌入模型、减少检索数量、使用 `llm` 推理、或采用流式输出来改善用户体验。对于入门阶段来说，流式输出是投入产出比最高的优化 `llm` 用户能看到文字逐字出现，等待的焦虑感会大幅降低。